SELENIUM HYBRID FRAMEWORK

Complete Learning Notes: Page Object Model, PageFactory, Java and TestNG

Beginner-friendly explanations • Corrected code • Real-world scenarios • Best practices • Hands-on exercises

---

Study basis

Built from the supplied “Selenium Hybrid Framework - POM Class 1” notes. Original concepts are preserved, and compile errors, conceptual inaccuracies, missing test assertions, lifecycle issues, and unsafe credential handling are called out explicitly.

Important security note

The supplied source file contained a hard-coded username and password. They are intentionally not reproduced in these notes. If they are real and still active, change the password and remove the credentials from shared files and any repository history.

Reference implementation assumes Java, Selenium 4, Maven and TestNG. UI locators must be checked against the current application DOM before execution.

How to use these notes

Read the concepts first, then trace the code in order: BaseTest -> LoginPage -> ContactPage -> Test class. Keep the source examples and the improved examples separate in your mind: the source is the starting point; the improved examples show how to make the design safer and easier to maintain.

Contents

- 1. Learning goals and framework overview
- 2. What Page Object Model solves
- 3. PageFactory, @FindBy, @FindBys and @FindAll
- 4. Locator strategy and DOM basics
- 5. Walkthrough of the supplied code
- 6. A cleaner project structure and Maven setup
- 7. Improved BaseTest: WebDriver lifecycle and waits
- 8. Improved LoginPage and login tests
- 9. Improved ContactPage and form handling
- 10. TestNG lifecycle, priorities and assertions
- 11. Real-world test scenarios
- 12. Data-driven tests and environment configuration
- 13. Common exceptions and troubleshooting
- 14. Framework best practices and mistakes to avoid
- 15. Hands-on assignments with expected outcomes
- 16. Interview questions and quick revision sheet
- 17. Glossary and official references

The key idea

Tests should describe the business behaviour being verified. Page objects should know how to interact with the page. Test data and secrets should be supplied separately. Assertions belong primarily in test classes.

1. Learning goals and framework overview

After working through this document, a beginner should be able to explain why a Page Object Model (POM) is used, create a page class, declare locators with @FindBy, initialize fields with PageFactory, write reusable page methods, manage a WebDriver session with TestNG, wait for dynamic UI elements, and verify outcomes with assertions.

1.1 What is a Selenium automation framework?

Selenium WebDriver is the browser-automation API. A framework is the organized set of code, conventions, configuration and support utilities around tests. A framework usually decides where tests live, how browsers start and stop, how page elements are located, how test data is supplied, how failures are reported, and how tests run locally or in CI.

Layer	Responsibility	Example from this lesson
Base / test setup	Create and close the browser; set shared configuration.	BaseTest
Page layer	Store page locators and expose page actions.	LoginPage, ContactPage
Test layer	Run a user scenario and assert the outcome.	LoginPageTest, ContactPageTest
Test data / configuration	Provide URL, credentials and scenario data.	Environment variables, system properties
Reporting / support (later additions)	Capture evidence and publish results.	TestNG reports, listeners, screenshots

1.2 Is this already a complete hybrid framework?

The supplied class focuses mainly on POM with PageFactory and TestNG. In many teams, “hybrid framework” means a combination of techniques such as POM, data-driven testing, reusable utilities, configuration management and reporting. The supplied notes are a useful starting layer, but they do not yet implement all of those pieces.

1.3 Typical execution flow

1. TestNG selects a test method.
2. A setup method creates a new WebDriver session and opens the target URL.
3. The test constructs a page object and passes it the driver.
4. The page object initializes its element fields and performs user-facing actions.
5. The test checks an observable result with an assertion.
6. A teardown method closes the browser even if the test fails.

2. What Page Object Model solves

Page Object Model is a design pattern in which each page or meaningful page component is represented by a class. That class owns the locators and the actions relevant to that page. Tests call meaningful methods such as loginAs(...) or createContact(...), instead of repeating low-level Selenium commands in every test.

2.1 Without POM

```
driver.findElement(By.id("email")).sendKeys(username);
driver.findElement(By.id("password")).sendKeys(password);
driver.findElement(By.xpath("//button[normalize-space()='Login']")).click();
```

If ten tests use the same login button and the application changes its markup, you may have to edit ten tests. The tests also become harder to read because business intent is mixed with browser mechanics.

2.2 With POM

```
LoginPage loginPage = new LoginPage(driver);
loginPage.loginAs(username, password);
```

The page object owns the details of how login works. If a locator changes, the usual repair is made in one page class. The test stays focused on the scenario and expected behaviour.

2.3 Benefits and boundaries

- Maintenance: centralize locators and page actions.
- Readability: express user actions in business language.
- Reuse: use the same page object from login, contact, smoke and regression tests.
- Separation of concerns: tests decide what should be true; page objects know how to interact with the UI.
- Not a magic stability fix: poor locators, missing waits, shared test data and weak assertions can still make a POM framework flaky.

Design rule from Selenium guidance

Page objects generally should not contain test assertions. Keep assertions in the test layer, with the limited exception of a page object checking that it has opened on the expected page.

3. PageFactory, @FindBy, @FindBys and @FindAll

3.1 What is PageFactory?

PageFactory is a Selenium support utility that initializes WebElement fields declared in a page class. When using annotations, PageFactory reads the locator definition and creates element proxies. The standard initialization call is PageFactory.initElements(driver, this). The supplied source contains a typo in one early snippet: “initiElements” is not the correct method name.

```
public class LoginPage {
   @FindBy(id = "email")
   private WebElement username;

   public LoginPage(WebDriver driver) {
      PageFactory.initElements(driver, this);
   }
}
```

The constructor runs when the test creates the page object. The driver tells PageFactory which browser session to use; this refers to the current LoginPage object whose fields are being initialized.

3.2 @FindBy

@FindBy declares how Selenium should locate an element. It is commonly used for a single WebElement, but it can also be used with List<WebElement> when the locator matches multiple elements.

```
@FindBy(id = "email")
private WebElement username;

@FindBy(css = ".contact-row")
private List<WebElement> contactRows;
```

In the first field, username represents the email input. In the second, contactRows represents a collection of matching rows. A single WebElement field is not the same as a list of every match.

3.3 @FindBys: important correction

Correction to the supplied class notes

@FindBys does not simply mean “locate multiple elements.” It combines locator criteria as a chained search: each subsequent criterion is applied within the result/context of the previous criterion. Use List<WebElement> when the goal is to collect multiple matching elements.

```
@FindBys({
   @FindBy(css = "form#contact-form"),
   @FindBy(css = "input[name='email']")
})
private WebElement contactEmailInput;
```

Conceptually, this searches for an email input inside the contact form. This example assumes the application has a form with id contact-form and an input with name=email; adapt it to the real DOM. Because @FindBys is a chained lookup, the structure of the DOM matters.

3.4 @FindAll

@FindAll is used when alternative locator definitions can identify an element. It represents alternatives (OR-style matching), unlike the chained lookup behaviour of @FindBys.

```
@FindAll({
   @FindBy(id = "email"),
   @FindBy(name = "username")
})
private WebElement usernameField;
```

This is useful only if the application genuinely uses one of those alternative attributes across supported page variants. Prefer one stable locator wherever possible; multiple alternatives can hide inconsistency if overused.

3.5 PageFactory and lazy element lookup

PageFactory generally creates proxies and resolves the element when a WebElement operation is invoked. This means initialization does not guarantee that the element is already visible or clickable. Wait for the condition needed before interacting. Avoid @CacheLookup for dynamic elements unless you understand the stale-element implications.

3.6 Encapsulation and visibility

Declare page fields private unless there is a deliberate reason to expose them. Tests should call methods such as enterUsername(...) rather than directly reaching into username.sendKeys(...). This keeps page internals contained and allows selectors or implementation details to change without rewriting tests.

4. Locator strategy and DOM basics

A locator identifies an element in the Document Object Model (DOM). The browser DOM is a tree of HTML elements. Selenium uses a locator strategy to find a WebElement, then commands such as click(), sendKeys(), clear(), getText() or isDisplayed() interact with it.

Strategy	Example	Guidance
id	By.id("email")	Usually a strong first choice when unique and stable.
name	By.name("password")	Useful when the application supplies a

Strategy	Example	Guidance
		stable name attribute.
CSS selector	By.cssSelector("input[type=email]")	Compact and readable for many DOM structures.
XPath	By.xpath("//button[normalize-space()='Login']")	Powerful for text and relationships; keep it readable.
linkText	By.linkText("Contacts")	Works for anchor/link elements with matching visible text.
className	By.className("primary-button")	Avoid compound class strings; classes may be unstable.
tagName	By.tagName("button")	Often matches many elements; usually needs more context.

4.1 Why stable locators matter

A locator tied to a generated class such as _nav_3as89_1 can break after a front-end build changes even if the page still looks the same. When available, prefer stable ids, names, accessible roles/labels, or dedicated test attributes such as data-testid. Use the shortest unique locator that describes the intended element.

4.2 Improve the source locators

Source-style locator	Risk	Possible improvement
//nav[@class='_nav_3as89_1']/ descendant::span[text()='Contacts']	Depends on a generated class and exact text structure.	Prefer a stable data-testid or a stable link/button accessible name.
//button[text()='Login']	Fails if whitespace or nested markup changes.	Use //button[normalize-space()='Login'] when text is the correct contract.
//textarea[@class='_textarea_1hcox_39 ']	Generated class and trailing space make it brittle.	Use id, name, label association or data-testid if available.
//input[@placeholder='Email']	Placeholder can be edited for UX or localization.	Use a stable id/name/label/test attribute where possible.

4.3 How to inspect the DOM

1. Open the application in Chrome and right-click the field or button.
2. Choose Inspect to locate the corresponding HTML element in DevTools.
3. Look for id, name, type, aria-label, role, stable data attributes, and nearby parent containers.
4. Test a selector in DevTools Console (for CSS, document.querySelector(...); for XPath, use $x("...")) before writing it in Java.
5. Confirm the selector matches exactly the intended control, not a hidden template or duplicate element.

4.4 Example DOM and matching selector

```
<form id="login-form">
  <label for="email">Email</label>
  <input id="email" name="email" type="email" />
  <label for="password">Password</label>
  <input id="password" name="password" type="password" />
  <button type="submit">Login</button>
</form>
```

For this sample DOM, @FindBy(id = 'email') targets the email input, @FindBy(id = 'password') targets the password input, and @FindBy(xpath = "//button[normalize-space()='Login']") targets the submit button. If an application uses different attributes, the locators must be updated accordingly.

5. Walkthrough of the supplied code

5.1 BaseTest in the source

```
public class BaseTest {
    public static WebDriver driver;

    public static void initialization() {
        driver = new ChromeDriver();
        driver.get("https://ui.freecrm.com/login");
        driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(30));
        driver.manage().timeouts().pageLoadTimeout(Duration.ofSeconds(30));
        driver.manage().window().maximize();
    }
}
```

What it does: declares a WebDriver, starts Chrome, navigates to the login URL, configures timeouts, and maximizes the browser. It acts as a shared setup class in the source notes.

What needs improvement: a static shared driver can cause tests to interfere with each other, especially when tests are parallelized. A 30-second implicit wait applies to every element lookup; the notes below use explicit waits instead. The URL should be configuration, not hard-coded into test infrastructure. The browser must be closed reliably in teardown.

5.2 LoginPage in the source

The source declares fields for username, password and the Login button, initializes them in the constructor with PageFactory.initElements(driver, this), then exposes methods that accept test data. This is the correct responsibility for a page object: it knows the page controls and how to act on them.

Compile and consistency issues in the first rough snippet include a misspelled PageFactory method, missing Java parameter types, missing semicolons, missing imports, inconsistent use of private fields and package syntax, and a typo in the package declaration. Treat that snippet as classroom pseudocode rather than a ready-to-run Java class.

5.3 ContactPage in the source

ContactPage centralizes the Contacts link, Create button, first-name and last-name fields, email, description, category and Save button. It uses Select to choose a category by visible text, then offers four overloaded createContact(...) methods with different parameter counts.

The example demonstrates method overloading: Java picks a method based on its parameter list. However, the four overloads duplicate navigation, field entry and save logic. If the sequence changes, all four methods could need edits. A single method with optional fields or a small Contact test-data object is generally easier to maintain.

5.4 The test classes in the source

LoginPageTest calls initialization, creates LoginPage, enters credentials and clicks Login. However, no assertion verifies that login succeeded, so the test may pass even when the application displays an error. Teardown calls quit() only in that test class.

ContactPageTest calls Thread.sleep(7000), creates ContactPage and submits contact data, but it does not initialize or close its own driver. It implicitly relies on some other test having initialized a shared static driver, and that other test may already have quit the browser. Test execution order should never be relied on this way. Every test should receive a valid browser fixture and clean up after itself.

Most important defects to fix first

1) remove hard-coded credentials; 2) pass WebDriver into page objects; 3) make each test independent; 4) replace Thread.sleep with explicit waits; 5) assert the expected outcome; 6) remove duplicated contact-creation logic.

6. A cleaner project structure and Maven setup

6.1 Recommended folder structure

```
selenium-hybrid-framework/
├── pom.xml
├── src/
│   └── test/
│       ├── java/
│       │   ├── base/
│       │   │   └── BaseTest.java
│       │   ├── pages/
│       │   │   ├── LoginPage.java
│       │   │   └── ContactPage.java
│       │   └── tests/
│       │       ├── LoginPageTest.java
│       │       └── ContactPageTest.java
│       └── resources/
│           └── testng.xml
```

The base package owns test setup/teardown, pages owns page abstractions, tests contains scenarios and assertions, and resources holds suite/configuration files. Bigger projects can add config readers, test-data factories, listeners, screenshot utilities, API setup, reporting and reusable components.

6.2 Example pom.xml

The versions below are a practical example based on official releases available around the date of writing. Use versions approved by your project, and update dependencies deliberately rather than blindly using “latest”.

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>selenium-hybrid-framework</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <maven.compiler.release>17</maven.compiler.release>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <selenium.version>4.51.0</selenium.version>
    <testng.version>7.12.0</testng.version>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.seleniumhq.selenium</groupId>
      <artifactId>selenium-java</artifactId>
      <version>${selenium.version}</version>
    </dependency>
    <dependency>
      <groupId>org.testng</groupId>
      <artifactId>testng</artifactId>
      <version>${testng.version}</version>
      <scope>test</scope>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-surefire-plugin</artifactId>
        <version>3.5.4</version>
      </plugin>
    </plugins>
  </build>
</project>
```

Selenium Manager is included in modern Selenium releases and can often manage browser drivers automatically. The browser itself must still be installed and compatible with the environment. In locked-down CI environments, follow the organization’s browser/driver provisioning process.

6.3 Import hygiene

Use explicit imports instead of relying on unexplained IDE auto-imports. Common classes in these notes include WebDriver, WebElement, ChromeDriver, PageFactory, FindBy, Select, WebDriverWait, ExpectedConditions, Duration, TestNG annotations and Assert.

7. Improved BaseTest: WebDriver lifecycle and waits

This improved example uses an instance driver and creates a browser per test method. It configures a page-load timeout and leaves implicit wait at its default zero so explicit waits can be used consistently. Replace the sample default URL with a URL for a test environment.

```
package base;

import java.time.Duration;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.chrome.ChromeOptions;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;

public abstract class BaseTest {
   protected WebDriver driver;

   @BeforeMethod
   public void setUp() {
      ChromeOptions options = new ChromeOptions();
      // In CI, enable headless mode when appropriate:
      // options.addArguments("--headless=new");

      driver = new ChromeDriver(options);
      driver.manage().window().maximize();
      driver.manage().timeouts().pageLoadTimeout(Duration.ofSeconds(30));

      String baseUrl = System.getProperty(
         "baseUrl", "https://your-test-app.example/login");
      driver.get(baseUrl);
   }

   @AfterMethod(alwaysRun = true)
   public void tearDown() {
      if (driver != null) {
         driver.quit();
      }
   }
}
```

7.1 Line-by-line explanation

- protected WebDriver driver: available to subclasses but not globally shared as a static variable.
- @BeforeMethod: TestNG calls setup before each test method, giving each test a clean browser session.
- new ChromeDriver(options): starts Chrome. Selenium Manager may locate or manage the driver in supported environments.
- pageLoadTimeout: limits how long navigation waits for a page-load event; it does not wait for every AJAX call or UI widget.
- System.getProperty("baseUrl", ...): reads a Maven/system property and falls back to a clearly marked example URL.
- @AfterMethod(alwaysRun = true): asks TestNG to run cleanup even after test failures or skips where applicable.
- driver.quit(): closes all windows for that WebDriver session. driver.close() only closes the current window.
- Prefer explicit waits for specific conditions. Selenium warns against mixing implicit and explicit waits because their combined timing can be unpredictable.

7.2 Run with a configured URL

```
mvn test -DbaseUrl=https://your-test-environment.example/login
```

Use a test or staging environment, not a production account. Keep the exact base URL consistent with the environment under test. For systems requiring SSO or a complex login bootstrap, the setup may need additional steps.

8. Improved LoginPage and login tests

8.1 LoginPage.java

```
package pages;

import java.time.Duration;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.FindBy;
import org.openqa.selenium.support.PageFactory;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;

public class LoginPage {
    private final WebDriver driver;
    private final WebDriverWait wait;

    @FindBy(id = "email")
    private WebElement username;

    @FindBy(id = "password")
    private WebElement password;

    @FindBy(xpath = "//button[normalize-space()='Login']")
    private WebElement loginButton;

    @FindBy(css = "[role='alert']")
    private WebElement errorMessage;

    public LoginPage(WebDriver driver) {
        this.driver = driver;
        this.wait = new WebDriverWait(driver, Duration.ofSeconds(10));
        PageFactory.initElements(driver, this);
    }

    public void enterUsername(String value) {
        wait.until(ExpectedConditions.visibilityOf(username));
        username.clear();
        username.sendKeys(value);
    }

    public void enterPassword(String value) {
        wait.until(ExpectedConditions.visibilityOf(password));
        password.clear();
        password.sendKeys(value);
    }

    public void clickLogin() {
        wait.until(ExpectedConditions.elementToBeClickable(loginButton)).click();
    }

    public void loginAs(String user, String pass) {
        enterUsername(user);
        enterPassword(pass);
        clickLogin();
    }

    public String getErrorMessage() {
        return wait.until(ExpectedConditions.visibilityOf(errorMessage)).getText();
    }
}
```

8.2 Explanation of the important parts

Code element	What it means
private final WebDriver driver	The page object keeps the browser session it was given.
WebDriverWait(driver, Duration.ofSeconds(10))	Waits up to 10 seconds for a requested condition, polling repeatedly.
@FindBy(id = "email")	Maps the username field to the element whose id is email.
PageFactory.initElements(driver, this)	Initializes the annotated page fields for this driver.
visibilityOf(username)	Waits until the element exists and is visible before interacting.
username.clear()	Removes existing text so a repeated action does not append to old input.
elementToBeClickable(loginButton)	Waits until the button is visible and enabled enough to click.
loginAs(user, pass)	Combines lower-level actions into a meaningful page service.

The [role=alert] locator is illustrative. If the application uses a different element for login errors, inspect the DOM and replace it. A page object should not assume the error markup without checking the application.

8.3 Test code: valid login

```
package tests;

import org.testng.Assert;
import org.testng.annotations.Test;
import base.BaseTest;
import pages.LoginPage;

public class LoginPageTest extends BaseTest {
  @Test
  public void validLoginShouldOpenDashboard() {
    String username = System.getenv("APP_USERNAME");
    String password = System.getenv("APP_PASSWORD");
    Assert.assertNotNull(username, "Set APP_USERNAME before running tests");
    Assert.assertNotNull(password, "Set APP_PASSWORD before running tests");

    LoginPage loginPage = new LoginPage(driver);
    loginPage.loginAs(username, password);

    // Replace this example check with a stable, app-specific assertion.
    Assert.assertTrue(
      driver.getCurrentUrl().contains("dashboard"),
      "Expected successful login to navigate to the dashboard");
  }
}
```

The URL assertion is an example, not a universal truth. Some applications keep the same route and update the page in place. In that case, assert a stable dashboard heading, logged-in user menu, or another reliable success indicator instead. The test is meaningful only when the assertion checks the application's defined success condition.

8.4 Test code: invalid login

```
@Test
public void invalidLoginShouldShowError() {
    LoginPage loginPage = new LoginPage(driver);
    loginPage.loginAs("invalid-user@example.com", "incorrect-password");

    Assert.assertEquals(
        loginPage.getErrorMessage(),
        "Invalid username or password");
}
```

The expected message must match the application contract exactly, or use a contains assertion if the message includes dynamic text. Never use a real user password in a negative test.

9. Improved ContactPage and form handling

9.1 ContactPage.java

This implementation keeps the actions in one place and removes four duplicated overloads. It assumes the category control is a native HTML <select> element. If the UI uses a custom JavaScript dropdown, Selenium's Select class will not work; use the application-specific click-and-select interaction instead.

```
package pages;
import java.time.Duration;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.FindBy;
import org.openqa.selenium.support.PageFactory;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.Select;
import org.openqa.selenium.support.ui.WebDriverWait;


public class ContactPage {
   private final WebDriverWait wait;
   @FindBy(xpath = "//nav[@class='_nav_3as89_1']/descendant::span[normalize-space()='Contacts']")
   private WebElement contactsLink;
   @FindBy(xpath = "//button[normalize-space()='Create']")
   private WebElement createButton;
   @FindBy(id = "first-name")
   private WebElement firstNameField;
   @FindBy(id = "last-name")
   private WebElement lastNameField;
   @FindBy(css = "input[placeholder='Email']")
   private WebElement emailField;
   @FindBy(xpath = "//textarea")
   private WebElement descriptionField;
   @FindBy(id = "category")
   private WebElement categorySelect;
   @FindBy(xpath = "//button[normalize-space()='Save']")
   private WebElement saveButton;
   public ContactPage(WebDriver driver) {
      this.wait = new WebDriverWait(driver, Duration.ofSeconds(10));
      PageFactory.initElements(driver, this);
   }

   public void openCreateContactForm() {
      wait.until(ExpectedConditions.elementToBeClickable(contactsLink)).click();
      wait.until(ExpectedConditions.elementToBeClickable(createButton)).click();
      wait.until(ExpectedConditions.visibilityOf(firstNameField));
   }

   public void createContact(String firstName, String lastName,
                             String email, String description,
                             String categoryName) {
      openCreateContactForm();
      firstNameField.sendKeys(firstName);
      lastNameField.sendKeys(lastName);
      if (email != null && !email.isBlank()) {
         emailField.sendKeys(email);
      }
      if (description != null && !description.isBlank()) {
         descriptionField.sendKeys(description);
      }
      if (categoryName != null && !categoryName.isBlank()) {
         new Select(categorySelect).selectByVisibleText(categoryName);
      }
      wait.until(ExpectedConditions.elementToBeClickable(saveButton)).click();
   }
}
```

9.2 What the method is doing

1. Click the Contacts navigation item and wait for it to be clickable.
2. Click Create and wait until the first-name field is visible, indicating that the form opened.
3. Enter required first and last names.
4. Enter email and description only when non-empty values were supplied.
5. Choose a category by visible text if a category was provided and if the element is a real HTML select.
6. Wait for the Save button to be clickable and submit the form.

This page method performs the action. It does not decide whether creation succeeded. The test must verify that a success notification appears, the contact appears in the list, or the application returns another defined success signal.

9.3 Why one method is usually cleaner than many overloads

Approach	Trade-off
Four overloaded methods, as in the source	Repeated navigation and submission code; fixes must be copied to several methods.
One method with optional fields	Less duplication; null/blank policy must be explicit.
Contact test-data object	Best when a contact has many fields or test scenarios; introduces a small model class.

9.4 Optional test-data model for larger projects

```
package testdata;

public class ContactData {
   private final String firstName;
   private final String lastName;
   private final String email;
   private final String description;
   private final String category;

   public ContactData(String firstName, String lastName, String email,
                      String description, String category) {
      this.firstName = firstName;
      this.lastName = lastName;
      this.email = email;
      this.description = description;
      this.category = category;
   }

   public String getFirstName() { return firstName; }
   public String getLastName() { return lastName; }
   public String getEmail() { return email; }
   public String getDescription() { return description; }
   public String getCategory() { return category; }
}
```

A data object keeps scenario data together and makes test cases easier to read. In a production framework, consider records (Java 16+) for simple immutable test-data holders, while respecting your team's Java version and style standards.

9.5 ContactPage test example

```
package tests;

import org.testng.Assert;
import org.testng.annotations.Test;
import base.BaseTest;
import pages.LoginPage;
import pages.ContactPage;

public class ContactPageTest extends BaseTest {
  @Test
  public void userCanCreateContact() {
    String username = System.getenv("APP_USERNAME");
    String password = System.getenv("APP_PASSWORD");
    Assert.assertNotNull(username, "Set APP_USERNAME before running tests");
    Assert.assertNotNull(password, "Set APP_PASSWORD before running tests");

    new LoginPage(driver).loginAs(username, password);
    ContactPage contacts = new ContactPage(driver);
    contacts.createContact(
      "Suraj", "Mane", "suraj.test@example.com",
      "Automation-created test contact", "Lead");

    // Add an app-specific assertion here, for example:
    // Assert.assertTrue(contacts.isSuccessToastDisplayed());
    // or assert that the created contact is visible in the contact list.
  }
}
```

Do not leave the assertion commented out in a real test

The snippet shows where the assertion belongs, but a production test must verify the result. Otherwise the test only proves that the automation commands did not throw an exception; it does not prove that the contact was saved.

10. TestNG lifecycle, priorities and assertions

10.1 Common annotations

Annotation	When it runs	Typical use
@BeforeSuite	Once before the suite	Global suite-level setup, if truly needed.
@BeforeTest	Before test methods in a TestNG XML <test> block	Environment setup scoped to that XML block.
@BeforeClass	Once before the first test method in the class	Class-level preparation.
@BeforeMethod	Before each @Test method	Fresh browser/session for each test.
@Test	Runs a test method	Scenario and assertions.
@AfterMethod	After each @Test method	Browser cleanup, per-test evidence.
@AfterClass	After test methods in the class	Class-scoped cleanup.
@AfterSuite	After the suite	Suite-level cleanup.

The source uses @BeforeTest and @AfterTest in LoginPageTest. That can work for certain suite structures, but it does not mean “before and after every Java test method.” For browser-per-test isolation, @BeforeMethod and @AfterMethod are often the clearer choice.

10.2 What does @Test(priority = 2) mean?

priority is a TestNG ordering hint. It is not a business assertion and should not be used as a substitute for test independence. Tests that require another test to have already created a record are fragile: reruns, parallel execution, or running one test alone may break them. Set up the required state inside the test or with explicit fixtures/API setup.

10.3 Assertions: verify, do not merely execute

```
Assert.assertEquals(actualMessage, "Invalid username or password");
Assert.assertTrue(driver.getCurrentUrl().contains("dashboard"),
      "Expected successful login to navigate to dashboard");
Assert.assertFalse(errorBanner.isDisplayed(),
      "Error banner should not be visible after successful login");
```

An assertion compares actual behaviour with expected behaviour. Without one, TestNG can mark a test as passed if the method completes without an unhandled exception. Use meaningful assertion messages so failure reports explain what was expected.

10.4 Suggested naming conventions

- Test methods: validLoginShouldOpenDashboard, invalidLoginShouldShowError, userCanCreateContact.
- Page methods: enterUsername, clickLogin, openCreateContactForm, selectCategory.
- Page fields: usernameField, loginButton, saveButton; use names that reveal the element purpose.
- Avoid vague names such as test1(), click1() or element1.

11. Real-world test scenarios

11.1 Login module

Scenario	Test data	Expected result	Automation notes
Valid login	Valid account from test secret store	Dashboard or authenticated state appears.	Use a stable dashboard marker; never log the password.
Invalid password	Valid username + wrong password	Error is shown; user stays unauthenticated.	Assert exact or contract-defined message.
Empty username	Blank username + valid/blank password	Required-field validation appears.	Check client and server validation if both apply.
Empty password	Valid username + blank password	Password-required message appears.	Do not assume browser validation equals application validation.
Session/logout	Valid login then logout	Protected view is no longer accessible.	Also test direct navigation after logout if in scope.

11.2 Contact module

Scenario	Expected result	Important detail
Create contact with required fields	Record saved and shown in the list.	Use unique test data to prevent collision.
Create contact with all optional fields	Saved record includes email/description/category.	Verify persisted values, not only a toast.
Submit missing last name	Validation appears and record is not created.	Confirm form validation rule with product requirements.

Scenario	Expected result	Important detail
Invalid email format	Email validation blocks or flags input.	Browser-native validation may affect observed message.
Duplicate email/contact	Expected duplicate policy is enforced.	Do not assume duplicates are forbidden unless specified.
Category selection	Correct category is saved.	Select class works only for native select elements.

11.3 Reliability and resilience scenarios

- Slow page: ensure waits fail with a clear timeout rather than a blind sleep.
- Button temporarily disabled: wait for clickability rather than immediately clicking.
- UI rerender after typing: avoid caching dynamic elements; re-locate when necessary.
- Browser startup failure in CI: check installed browser, permissions, headless options, network restrictions and logs.
- Test rerun: ensure records are unique or cleaned up, so rerunning the same scenario does not fail due to leftover data.

11.4 What a complete assertion might look like

Suppose the application displays a toast after saving and then lists the newly created contact. A stronger test checks at least one reliable success condition and, for a critical workflow, verifies the new record and important saved fields. A green toast alone may not guarantee that every input was persisted correctly.

12. Data-driven tests and environment configuration

A hybrid framework often uses data-driven testing: the same test logic runs with multiple data sets. TestNG provides @DataProvider. The current source does not include a DataProvider, so the following is an extension that fits naturally after POM basics.

12.1 TestNG DataProvider example

```
package tests;

import org.testng.Assert;
import org.testng.annotations.DataProvider;
import org.testng.annotations.Test;

public class LoginValidationDataTest {
  @DataProvider(name = "invalidLoginData")
  public Object[][] invalidLoginData() {
    return new Object[][] {
      { "unknown-user@example.com", "wrong-password" },
      { "another-user@example.com", "wrong-password" }
    };
  }

  @Test(dataProvider = "invalidLoginData")
  public void invalidCredentialsShouldBeRejected(String username,
                                                 String password) {
    // Create LoginPage using the test's driver and call loginAs(...).
    // Assert the application's documented error response.
  }
}
```

This snippet focuses on the TestNG parameter mechanism. In a real suite, the test class should extend BaseTest or use an equivalent fixture and include a concrete assertion. Never put real passwords into source-controlled data providers.

12.2 Credentials and configuration

Keep secrets out of source code, screenshots, reports and repository history. The examples read APP_USERNAME and APP_PASSWORD from the process environment. In CI, create these as protected secret variables and limit who can read them. Do not print environment variables or include them in failure messages.

```
# macOS / Linux shell example
export APP_USERNAME="qa-user@example.com"
export APP_PASSWORD="replace-with-a-test-secret"
mvn test -DbaseUrl="https://your-test-environment.example/login"
```

For Windows PowerShell, use $env:APP_USERNAME = "..." and $env:APP_PASSWORD = "..." before running Maven. Prefer the CI platform's secret manager over a local .env file for shared pipelines. If a local .env file is used by a helper tool, add it to .gitignore and do not commit it.

12.3 Why test data should be independent

A test should prepare the data it needs and should be repeatable. Use unique identifiers such as a timestamp or UUID in emails when the application permits, or arrange cleanup through a supported API/fixture. Avoid assuming another test has already created a record.

13. Common exceptions and troubleshooting

Symptom / exception	Likely causes	Practical checks
NoSuchElementException	Wrong locator; element not loaded; wrong page; iframe/shadow DOM issue.	Inspect the DOM, verify navigation, check frame context and wait condition.
TimeoutException	Expected condition never became true within the wait.	Check locator, app response, visibility/enabled state and actual failure screenshot.
StaleElementReferenceException	Element was detached/re-rendered after it was located.	Re-locate the element; wait for the refreshed state; avoid caching dynamic controls.
ElementClickInterceptedException	Overlay, toast, spinner or another element covers the target.	Wait for overlays to disappear; inspect z-index/scroll position; avoid JavaScript click as the first fix.
InvalidSelectorException	Malformed XPath or CSS selector.	Test the selector in DevTools and review quotes/brackets.
SessionNotCreatedException	Browser/driver/version/environment mismatch or startup failure.	Check browser installation, Selenium Manager logs, permissions, options and container configuration.
NullPointerException for driver	Setup did not run, driver was not passed, or global state is null.	Ensure test fixture creates a driver and each page gets the same active driver.
Select throws unexpected tag name	Dropdown is not a native HTML select.	Inspect the tag; use click-and-select steps for custom dropdowns.

13.1 Debugging sequence

1. Read the first meaningful exception and identify the failing statement.
2. Confirm the browser is on the expected URL and the expected page has loaded.
3. Inspect the locator and confirm it matches the correct element in the live DOM.
4. Check whether the element is present, visible, enabled and unobstructed.
5. Replace a fixed sleep with a wait on the exact required condition.
6. Capture a screenshot and useful non-secret logs on failure.
7. Re-run the smallest failing test before running the whole suite.

14. Framework best practices and mistakes to avoid

14.1 Best practices checklist

- Use one page class per page or reusable page component, not necessarily one class for every URL if several URLs share a component.
- Pass WebDriver into page constructors; avoid a global static driver.
- Keep element fields private and expose meaningful page actions.
- Keep assertions in tests; return observed state from pages where needed.
- Prefer stable ids, names, accessible labels and test attributes over brittle generated CSS classes.
- Use explicit waits for the condition required at that point in the workflow.
- Use a new browser per test method when test isolation is more valuable than startup cost.
- Use @AfterMethod(alwaysRun = true) and check driver != null before quitting.
- Use unique or cleaned-up test data; make tests repeatable and order-independent.
- Keep URLs and non-secret settings configurable and keep credentials in secret storage.
- Use assertions that verify business outcomes, not merely that commands executed.
- Capture diagnostic evidence on failure but mask tokens, passwords, PII and other sensitive information.
- Run smoke tests in CI first, then broader regression suites; publish test results and failure artifacts.
- Pin dependency versions and update them intentionally after compatibility checks.

14.2 Mistakes to avoid

Mistake	Why it hurts	Better option
Thread.sleep(7000)	Always waits the full duration and can still be too short.	WebDriverWait with a relevant condition.
No assertions	Test can pass despite a business failure.	Assert the success/error state explicitly.
Static global driver	Tests can share or overwrite browser state.	Instance driver per test/worker.
Tests depend on priority/order	Single tests and parallel runs become unreliable.	Prepare state in each test or fixture.
Repeated overloaded workflows	Changes must be duplicated and can drift.	One reusable action method or test-data model.
Hard-coded credentials	Leaks secrets and makes environments inflexible.	Environment/CI secrets.
Mix implicit and explicit waits	Can produce surprising timeout durations.	Use explicit waits consistently.
Assertions inside page object for business outcomes	Blurs page behaviour and test intent.	Expose observed state; assert in test.
Overcomplicated XPath	Hard to debug and fragile to markup changes.	Stable attributes and shorter selectors.
Ignoring teardown on failure	Leaked browser sessions and resource usage.	Always-run cleanup and null check.

14.3 Production-level upgrades

- Configuration module: centralize base URL, browser, timeout and environment settings.
- Browser factory: support Chrome/Firefox/Edge and headless CI runs in one place.
- BasePage: optional shared wait/click/type helpers, only where they reduce duplication without hiding Selenium behaviour.
- TestNG listener: attach screenshots, browser logs and diagnostic context for failed tests.
- Reporting: generate readable HTML reports and retain artifacts in CI.
- Data management: create, verify and clean test records using supported APIs or fixtures when practical.
- Parallel execution: use isolated driver instances (often ThreadLocal only with deliberate design) and avoid shared mutable test data.

15. Hands-on assignments with expected outcomes

Complete these exercises in a test or demo application that you are authorized to automate. Since the source application's UI may have changed, always confirm the current locators before executing the examples.

Exercise 1: build LoginPage

Create a LoginPage class with username, password and login button fields. Initialize it with PageFactory. Add enterUsername, enterPassword and clickLogin methods. Add a loginAs method that composes the three steps.

Expected outcome: the test class no longer contains raw locator strings for the login fields.

Exercise 2: add a valid-login assertion

Write a TestNG test that gets credentials from environment variables, logs in, then asserts a stable dashboard element or route. Include a failure message that explains the expected result.

Expected outcome: invalid login will fail the test instead of being reported as a pass.

Exercise 3: replace Thread.sleep

Find a test that sleeps for seven seconds. Determine what state the test is waiting for and replace the sleep with WebDriverWait and the most appropriate ExpectedConditions condition.

Expected outcome: the test continues as soon as the target state is ready and fails with a clear timeout if it never becomes ready.

Exercise 4: simplify ContactPage

Refactor the four overloaded createContact methods into a single method accepting required and optional fields, or create a ContactData object. Keep navigation and field interactions inside the page class.

Expected outcome: only one method owns the "open form -> fill -> save" sequence.

Exercise 5: add contact verification

After Save, wait for the application's actual success signal and assert that it occurred. A stronger version searches the contact list for the newly created name/email and verifies the saved values.

Expected outcome: a failed or rejected save causes the test to fail.

Exercise 6: prove test independence

Run ContactPageTest alone from the IDE or Maven. Then run LoginPageTest and ContactPageTest in reverse order. Each test must create and close its own driver and prepare its own authentication state.

Expected outcome: no test depends on a static driver or another test's execution order.

Exercise 7: run from command line

```
mvn test -DbaseUrl=https://your-test-environment.example/login
```

Set APP_USERNAME and APP_PASSWORD in your shell or CI secret manager first. Check the Maven exit code and TestNG report. Never paste actual credentials into the notes or commit history.

16. Interview questions and quick revision sheet

Q. What is POM? A. A design pattern that models pages/components as classes, centralizes locators and exposes meaningful user actions.

Q. Why do we use PageFactory? A. To initialize annotated WebElement fields and make page classes more concise; it does not remove the need for waits.

Q. What does @FindBy do? A. Defines how to locate a single element or a list of elements.

Q. What is the difference between @FindBy and @FindBys? A. @FindBy specifies a locator; @FindBys chains locator criteria through nested search contexts. It does not simply mean a list.

Q. What is @FindAll used for? A. Alternative locator definitions that can match an element using OR-style matching.

Q. Why keep WebElements private? A. To encapsulate page internals and force tests to use meaningful page methods.

Q. Why pass WebDriver into page constructors? A. It makes the dependency explicit and avoids hidden global/static session state.

Q. Why is Thread.sleep discouraged? A. It waits a fixed duration regardless of whether the condition is ready and makes tests slower or flaky.

Q. Implicit vs explicit wait? A. Implicit wait is global for element lookup; explicit wait polls for a particular condition. Do not mix them casually.

Q. What is the difference between close() and quit()? A. close() closes the current window; quit() ends the entire WebDriver session and closes its windows.

Q. Where should assertions live? A. Primarily in the test layer; page objects expose actions and state.

Q. Why is @BeforeMethod useful? A. It performs setup before each test method, supporting isolated execution.

Q. What is wrong with a static WebDriver? A. Shared mutable state can break test isolation and parallel execution.

Q. When does Select work? A. When the target is a native HTML select element, not an arbitrary custom dropdown.

Q. Why does a test need an assertion? A. Without one, the test may pass after actions even when the application did not achieve the expected outcome.

Q. What makes a locator maintainable? A. It is unique, stable, readable, and tied to a meaningful application contract rather than incidental generated markup.

Quick revision checklist

- Page classes contain locators and page actions.
- PageFactory.initElements(driver, this) is spelled correctly and called with the active driver.
- @FindBys means chained lookup; List<WebElement> means collection.
- Each test has setup, meaningful action, assertion and cleanup.
- Use explicit waits rather than Thread.sleep.
- Never hard-code credentials.
- Never rely on test order or another test's browser session.
- Use selectors that are stable in the application DOM.
- Verify actual business outcomes after Save or Login.

17. Glossary and official references

Term	Beginner-friendly meaning
DOM	The browser's in-memory tree representation of the page HTML.
WebDriver	Selenium's API for controlling a browser.
WebElement	A reference to an element such as an input, button, row or link.
Locator	A rule Selenium uses to find one or more elements.
POM	A design pattern for representing pages/components as classes.
PageFactory	Selenium support utility for initializing annotated page fields.
Explicit wait	A wait for a specific condition until success or timeout.
TestNG	Java test framework providing @Test, setup/teardown annotations, data providers and reports.
Assertion	A check that compares actual behaviour to expected behaviour.
Test isolation	A test can run independently without requiring another test's state.
Flaky test	A test that passes or fails inconsistently without a meaningful product change.
Native select	An HTML <select> element supported by Selenium's Select class.

Official references

These links are external references for deeper study. The examples in this document are training templates; the live application DOM and your team's environment should be treated as the final authority for selectors, routes and expected messages.

- **Selenium: Page Object Models:** https://www.selenium.dev/documentation/test_practices/encouraged/page_object_models/
- **Selenium: Locator strategies:** https://www.selenium.dev/documentation/webdriver/elements/locators/
- **Selenium: Locator best practices:** https://www.selenium.dev/documentation/test_practices/encouraged/locators/
- **Selenium: Waiting strategies:** https://www.selenium.dev/documentation/webdriver/waits/
- **Selenium Java API: PageFactory:** https://www.selenium.dev/selenium/docs/api/java/org/openqa/selenium/support/PageFactory.html
- **Selenium Java API: FindBy:** https://www.selenium.dev/selenium/docs/api/java/org/openqa/selenium/support/FindBy.html
- **TestNG official documentation:** https://testng.org/documentation.html
- **Selenium releases:** https://github.com/SeleniumHQ/selenium/releases
- **TestNG releases:** https://github.com/testng-team/testng/releases